Solution: Two Sum

Problem Statement

You are given an array of integers `nums` and an integer `target`. Your task is to find two distinct indices `i` and `j` such that the sum of `nums[i]` and `nums[j]` is equal to the `target`. You can assume that each input will have exactly one solution, and you may not use the same element twice.

Examples

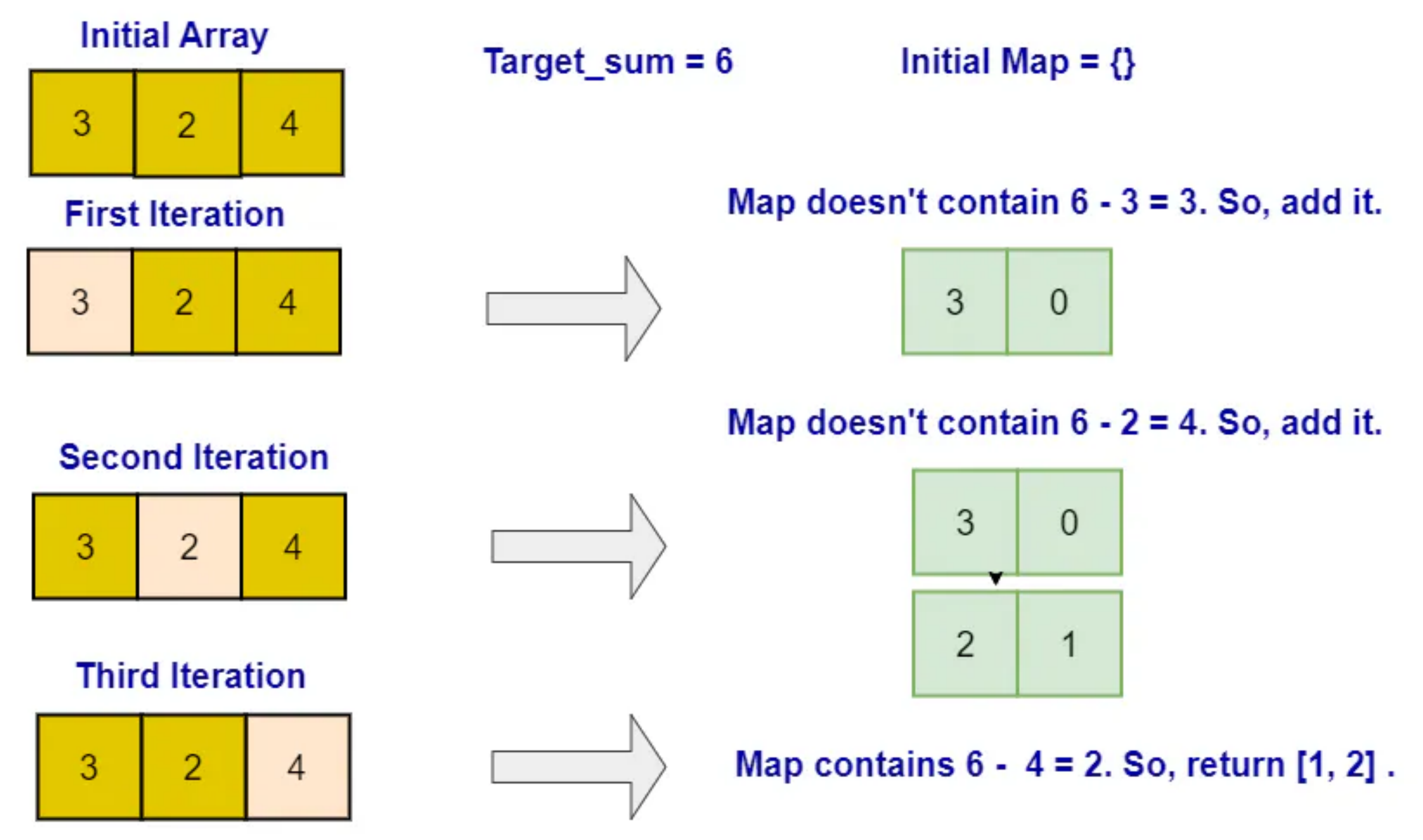
1. **Example 1**:
    - Input: `[3, 2, 4]`, `6`
    - Expected Output: `[1, 2]`
    - Justification: `nums[1] + nums[2]` gives `2 + 4` which equals `6`.
2. **Example 2**:
    - Input: `[-1, -2, -3, -4, -5]`, `-8`
    - Expected Output: `[2, 4]`
    - Justification: `nums[2] + nums[4]` yields `-3 + (-5)` which equals `-8`.
3. **Example 3**:
    - Input: `[10, 15, 20, 25, 30]`, `45`
    - Expected Output: `[1, 3]`
    - Justification: `nums[1] + nums[3]` gives `15 + 30` which equals `45`.

Solution

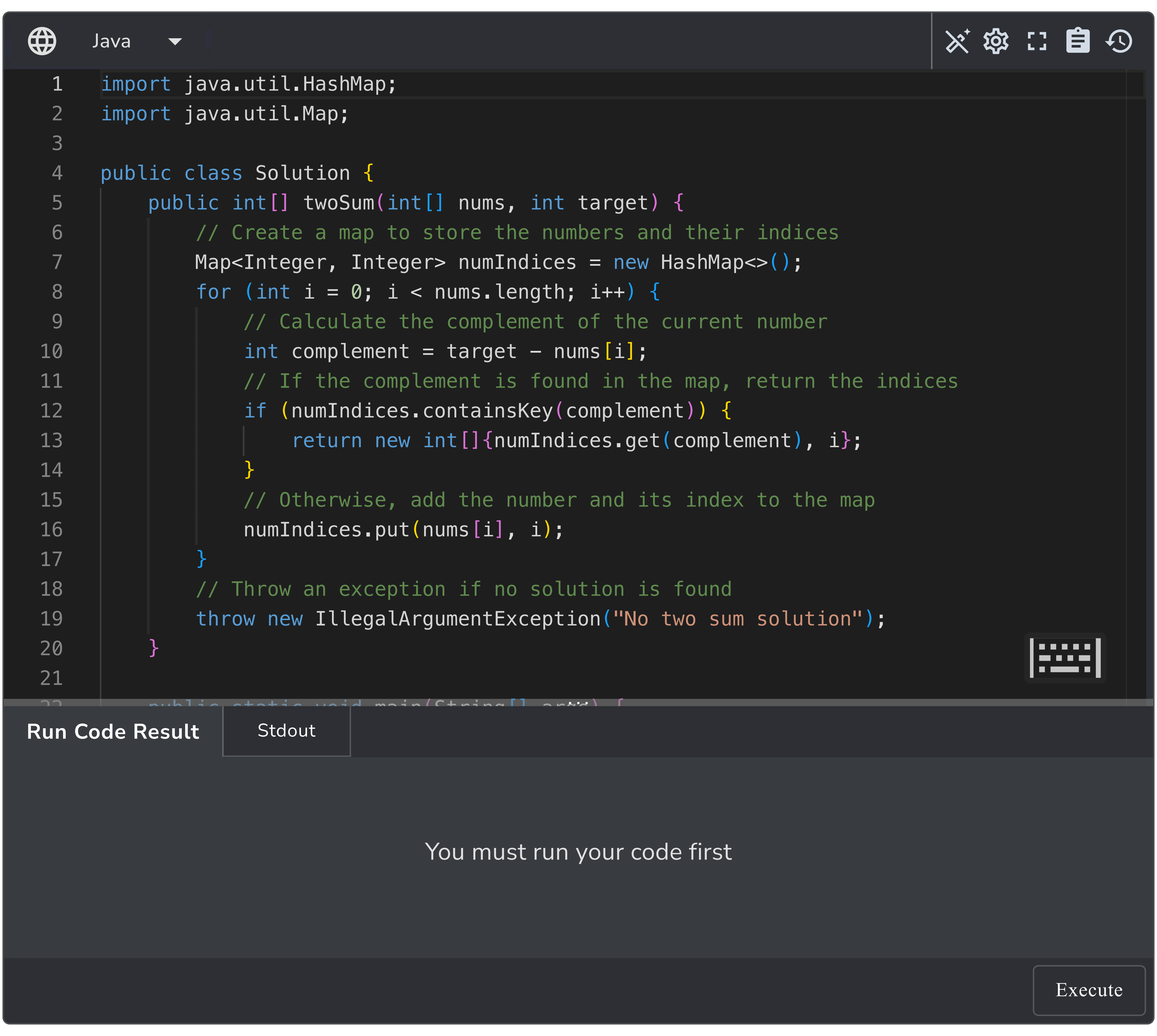
1. **Initialization**:
    - We start by creating a hashmap to store the elements of the array as we traverse it. The hashmap will store the number as the key and its index as the value. This will aid in quickly looking up the index of the complement of the current number (i.e., `target - nums[i]`).
2. **Traversal and Storage**:
    - We then traverse through the array. For each element, we calculate its complement by subtracting it from the target. We then check if this complement is present in our hashmap.
3. **Checking Complement**:
    - If the complement is found in the hashmap, it means we have found two numbers whose sum is equal to the target. We then return the indices of these two numbers as the result.
4. **Returning Result**:
    - If no such indices are found after traversing the entire array, we return an empty array, but since the problem guarantees one solution, this situation will not occur in valid inputs.

Algorithm Walkthrough

- Given the input array `[3, 2, 4]` and target `6`, we perform the following steps:
  1. **Step 1**: Initialize an empty hashmap `map`.
  2. **Step 2**: Traverse the array. The first element is `3`.
  3. **Step 3**: Calculate the complement: `6 - 3 = 3`. This is not found in `map`, so we add `3` with its index `0` to `map`.
  4. **Step 4**: Move to the next element `2`. Calculate the complement: `6 - 2 = 4`. This is not found in `map`, so we add `2` with its index `1` to `map`.
  5. **Step 5**: Move to the next element `4`. Calculate the complement: `6 - 4 = 2`. This is found in `map` at index `1`.
  6. **Step 6**: Since we have found two numbers whose sum is `6`, we return their indices `[1, 2]`.


Initial Array
3 2 4
Target_sum = 6
Initial Map = {}
First Iteration
3 2 4
Map doesn't contain 6 - 3 = 3. So, add it.
3 0
Second Iteration
3 2 4
Map doesn't contain 6 - 2 = 4. So, add it.
3 0
2 1
Third Iteration
3 2 4
Map contains 6 - 4 = 2. So, return [1, 2] .

Code



```java
import java.util.HashMap;
import java.util.Map;

public class Solution {
    public int[] twoSum(int[] nums, int target) {
        // Create a map to store the numbers and their indices
        Map<Integer, Integer> numIndices = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            // Calculate the complement of the current number
            int complement = target - nums[i];
            // If the complement is found in the map, return the indices
            if (numIndices.containsKey(complement)) {
                return new int[]{numIndices.get(complement), i};
            }
            // Otherwise, add the number and its index to the map
            numIndices.put(nums[i], i);
        }
        // Throw an exception if no solution is found
        throw new IllegalArgumentException("No two sum solution");
    }

```

You must run your code first

Complexity Analysis

- **Time Complexity**: The time complexity of this algorithm is O(n), where n is the number of elements in the array. This is because we traverse through the array once, and for each element, we perform O(1) operations to calculate the complement and check if it is in the hashmap.
- **Space Complexity**: The space complexity is also O(n), as in the worst case, we might have to store all the n elements in the hashmap.

← Previous
Two Sum (Easy)
Next →
Valid Perfect Square (Easy)
☐ Mark as Completed